SPRINT 5 · MATEMÁTICAS Y ESTADÍSTICA EN CÓDIGO · S05.1

NumPy y álgebra lineal

Los modelos de IA no son más que muchas operaciones con vectores y matrices. NumPy las hace sin bucles y a gran velocidad.

Los datos: la serie diaria de ocupación

`ocupacion_diaria.csv`: 1.096 días (2023–2025) del alojamiento de referencia de TurisData, con ocupación, temperatura, festivo y precio medio.

```
tabla = np.genfromtxt(dato("ocupacion_diaria.csv"), delimiter=",",
                      skip_header=1, usecols=(1, 2, 3, 4))
tabla.shape      # (1096, 4): filas = días, columnas = variables
```

Un **array** es una tabla de números del mismo tipo, con `shape`, `dtype` y `ndim`. Es mucho más rápido que una lista de Python.

Seleccionar sin bucles (cuaderno S05_01)

```
tabla[0, 1]              # un valor
tabla[:, 0]              # la columna 0 entera        → ocupacion
tabla[10:20, :2]         # filas 10-19, dos primeras columnas

festivos = festivo == 1
ocupacion[festivos].mean()                    # Ejercicio 2: media en festivos
ocupacion[(festivo == 0) & (precio > 100)] # máscara: & (y), | (o), ~ (no)
```

Una comparación devuelve `True/False` por elemento; usada como índice, deja solo los `True`.

Vectorizar y *broadcasting*

```
ingreso_diario = ocupacion / 100 * 120 * precio     # Ejercicio 3: 120 habitaciones
ocupacion_z = (ocupacion - ocupacion.mean()) / ocupacion.std()

tabla_z = (tabla - tabla.mean(axis=0)) / tabla.std(axis=0)   # Ejercicio 4
```

- **Vectorizar** = operar con todo el array a la vez.
- **Broadcasting**: NumPy «estira» el array pequeño. Restar a `(1096, 4)` un vector de 4 medias funciona directamente.
- `axis=0` calcula **por columna**; `axis=1` por fila.

Aleatorios reproducibles y media móvil

```
rng = np.random.default_rng(SEMILLA)                          # misma semilla → mismos números
simulada = np.clip(rng.normal(68.5, 9.6, 365), 0, 100)     # Ejercicio 5

media_movil = np.convolve(ocupacion, np.ones(7) / 7, mode="valid")   # mini-reto
```

> Fijar la semilla permite que el cliente (y tú mañana) repitáis el mismo resultado: **reproducibilidad**.

Álgebra lineal: vectores y matrices (cuaderno S05_02)

Producto escalar

`noches_dia @ precios` = noches × precio, sumado. Ejercicio 1: ingreso de un día.

Producto matricial

`(m×n) @ (n×k) = (m×k)`. Ejercicio 2: `noches_semana @ precios` da el ingreso de cada día.

Transpuesta e inversa

`A.T` cambia filas por columnas. A^{-1} solo existe si el determinante no es 0.

Sistemas de ecuaciones y regresión

```
precios_ab = np.linalg.solve(noches_ab, facturado)   # Ejercicio 3: precios perdidos

# Ecuación normal: la regresión lineal ES un sistema de ecuaciones
w = np.linalg.solve(X.T @ X, X.T @ ocupacion)        # Ejercicio 5
prediccion_25 = w[0] + w[1] * 25                     # ocupación a 25 °C
```

- `X` lleva una **columna de unos** para el término independiente.
- La pendiente sale **negativa** (más calor, menos ocupación). ¿Provoca el calor menos reservas? Lo discutimos en S05.3.

Ahora, a practicar

Lo que viene a continuación está en tu repositorio; no hace falta buscar ejercicios fuera.

CUADERNO **S05_01_numpy** — 35 min

Arrays, vectorización y *broadcasting*

sprints/S05/notebooks/S05_01_numpy.ipynb

CUADERNO **S05_02_algebra_lineal** — 35 min

Matrices, sistemas y ecuación normal

sprints/S05/notebooks/S05_02_algebra_lineal.ipynb